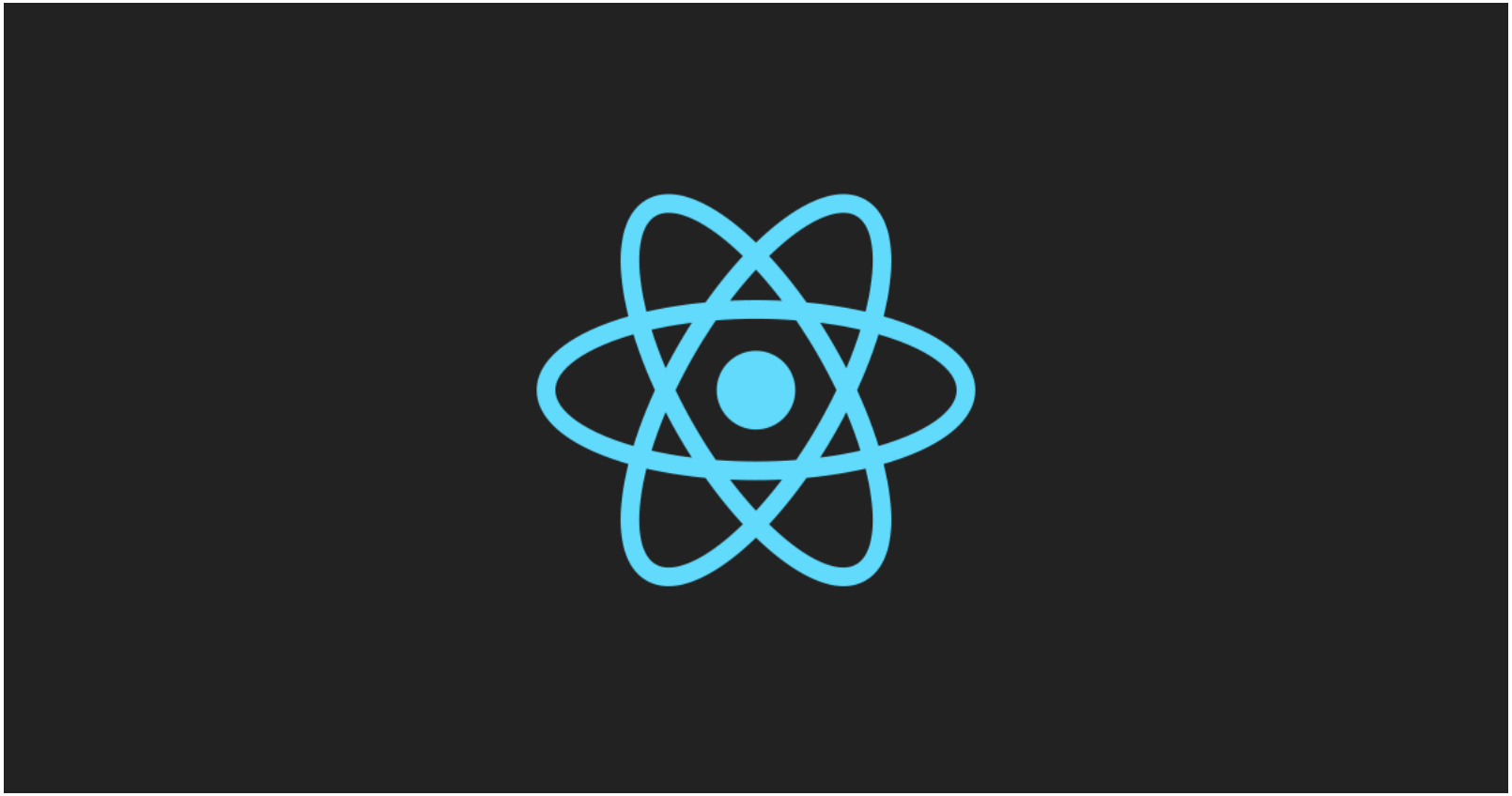
Client-side applications

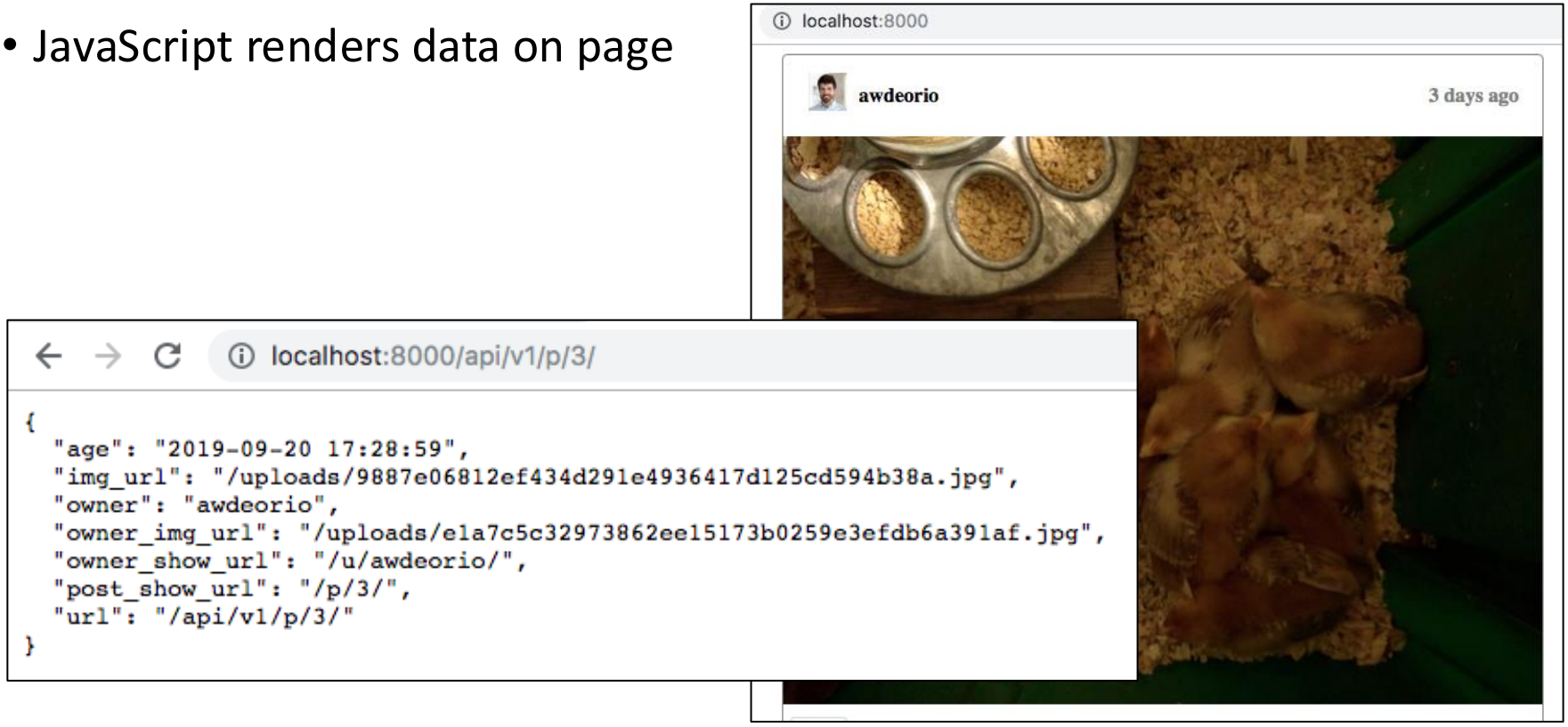


Agenda

- Review
- Client/server synchronization (read)
- Fetch API
- Promises
- Anonymous functions
- Client/server synchronization (write)

Review: REST APIs are for data exchange

- JavaScript runs in browser
- JavaScript receives JSON data from server
- JavaScript renders data on page



Review: Client-side dynamic pages

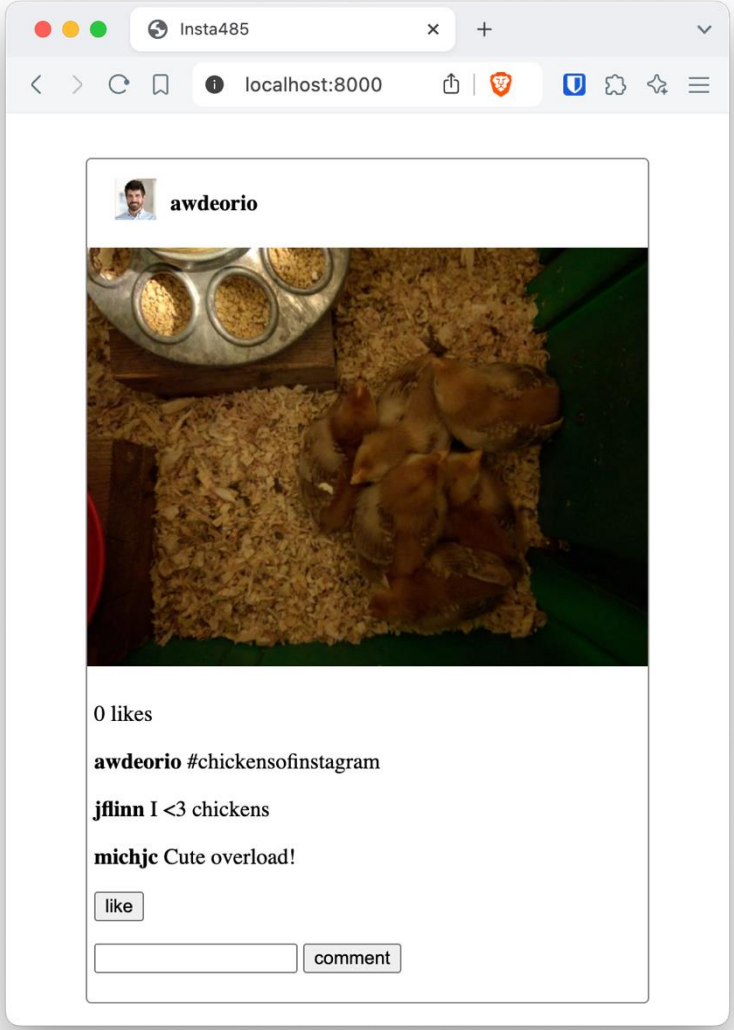
- **Problem**: Server code can't change the page after it has been generated and transferred to the client's browser

- **Solution**: Client-side dynamic pages - Code running in the client's web browser modifies the DOM. The rendered page changes.

Review: Client-side dynamic pages *no reload*

- With client-side dynamic pages, you can update just one part of the page

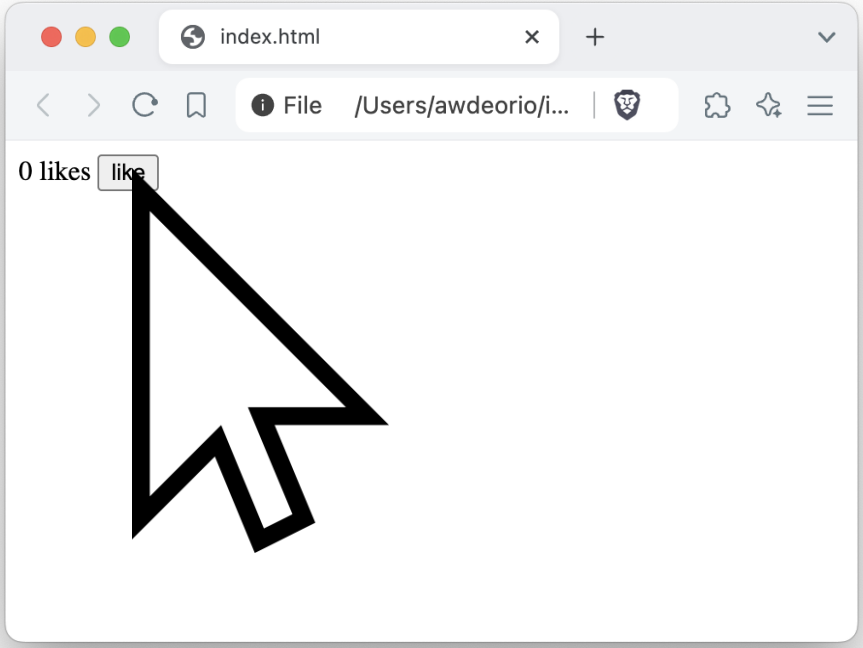
- Example: Project 3 like button



Last time

- Recall our like button
- **addLike()** runs when user clicks like button
- Event-driven programming

```
<html><body>
  <span id="numLikes">0</span> likes
  <button id="likeBtn">like</button>
  <script>
   let numLikes = 0;
   function addLike() {
     numLikes += 1;
     const n = document.getElementById('numLikes');
     n.textContent = numLikes;
   }
   const btn = document.getElementById("likeBtn");
   btn.addEventListener("click", addLike);
  </script>
</body></html>
```



Review: Event handler

- An *event handler* is a function
- An *event handler* runs after an *event* occurs
- AKA callback

```
<html><body>
  <span id="numLikes">0</span> likes
  <button id="likeBtn">like</button>
  <script>
   let numLikes = 0;
   function addLike() {
     numLikes += 1;
     const n = document.getElementById('numLikes');
     n.textContent = numLikes;
   }
   const btn = document.getElementById("likeBtn");
   btn.addEventListener("click", addLike);
  </script>
</body></html>
```

Review: Event table

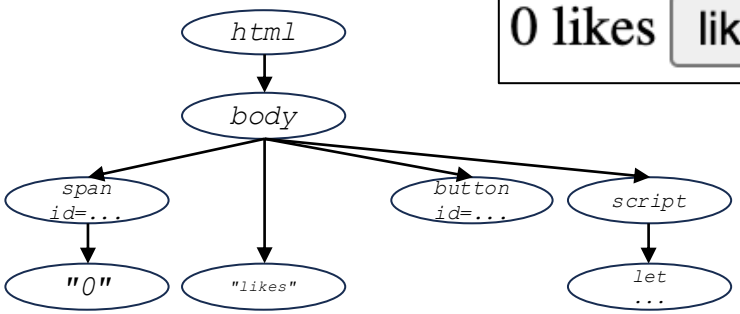
Event Table	
likeBtn **click**	**addLike**

- The *event table* maps *events* to *event handlers*
  - Nomenclature is not universal
- **`addEventListener()`** updates the event table

```
<html><body>
  <span id="numLikes">0</span> likes
  <button id="likeBtn">like</button>
  <script>
   let numLikes = 0;
   function addLike() {
     numLikes += 1;
     const n = document.getElementById('numLikes');
     n.textContent = numLikes;
   }
   const btn = document.getElementById("likeBtn");
   btn.addEventListener("click", addLike);
  </script>
</body></html>
```

Review: One button click

DOM



Rendered Page

```
<html><body>
  <span id="numLikes">0</span> likes
  <button id="likeBtn">like</button>
  <script>
   let numLikes = 0;
   function addLike() {
     numLikes += 1;
     const n = document.getElementById('numLikes');
     n.textContent = numLikes;
   }
   const btn = document.getElementById("likeBtn");
   btn.addEventListener("click", addLike);
  </script>
</body></html>
```

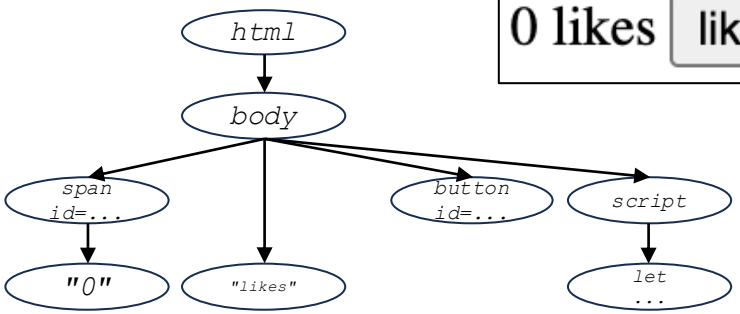
Event Table	
likeBtn **click**	**addLike**

Event Queue	*head*

Stack

Review: One button click

DOM



Rendered Page

```
<html><body>
  <span id="numLikes">0</span> likes
  <button id="likeBtn">like</button>
  <script>
   let numLikes = 0;
   function addLike() {
     numLikes += 1;
     const n = document.getElementById('numLikes');
     n.textContent = numLikes;
   }
   const btn = document.getElementById("likeBtn");
   btn.addEventListener("click", addLike);
  </script>
</body></html>
```

Event Table	
likeBtn **click**	**addLike**

Event Queue	*head*

Stack

What happens when the page is reloaded?

Server-side vs. client-side dynamic pages

Server-side dynamic pages

- Server response is the output of a function
- Good for database access
  - Server function runs SQL query
- Bad for user interaction
  - Refresh required

Client-side dynamic pages

- JavaScript running in the client's web browser modifies the DOM
- Bad for database access
  - JS runs on client, not server
- Good for user interaction
  - Modify the DOM without refresh

Can we have the best of both worlds?

Synchronize client and server

- Goal: synchronize client and server

- Server keeps track of the database
- Client updates the user interface
- Client and server communicate via a *REST API*

Agenda

- Review
- **Client/server synchronization (read)**
- Fetch API
- Promises
- Anonymous functions
- Client/server synchronization (write)

Likes REST API

- **Problem**: Client like count resets on page load
- **Solution**: Server stores the like count, access w/ REST API

```
LIKES = 3 # HACK "Database"

@app.route('/api/v1/likes/')
def get_likes():
    """Return num likes."""
    context = {"numLikes": LIKES}
    return flask.jsonify(context), 200
```

```
$ GET /api/v1/likes/
{
    "numLikes": 3
}
```

Out of sync

- Number of likes is now stored in two places
  - Client, server
- **Problem**: Client and server are out of sync
- **Solution**: Client reads num likes from REST API

```
LIKES = 3 # HACK "Database"

@app.route('/api/v1/likes/')
def get_likes():
    """Return num likes."""
    context = {"numLikes": LIKES}
    return flask.jsonify(context), 200
```

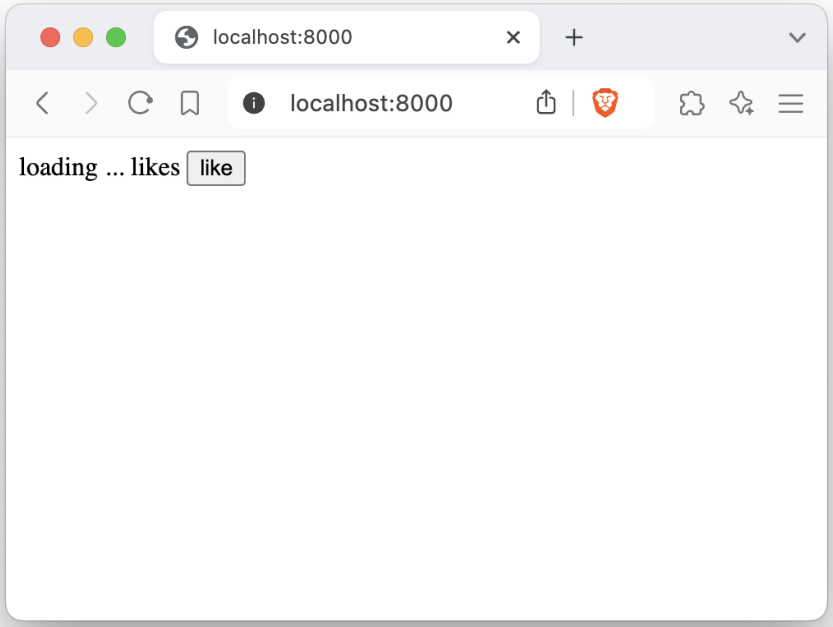
Agenda

- Review
- Client/server synchronization (read)
- **Fetch API**
- Promises
- Anonymous functions
- Client/server synchronization (write)

Goal

1. "loading ..." placeholder
2. Read initial `numLikes` from server REST API
3. Update the page
   - Replace "loading ..."

```
<html><body>
  <span id="numLikes">loading ...</span> likes
  <button id="likeBtn">like</button>
  <script>
   let numLikes = 0;
   function addLike() {
     numLikes += 1;
     const n = document.getElementById('numLikes');
     n.textContent = numLikes;
   }
   const btn = document.getElementById("likeBtn");
   btn.addEventListener("click", addLike);

   // Read REST API here
  </script>
</body></html>
```



Fetch API

- **Problem**: How to call a REST API from a JavaScript program?
- **Solution**: Fetch API

- The Fetch API is a JavaScript function for making HTTP requests
- The Fetch API is built into the browser/interpreter

Minimal example

```
<html><body>
  <span id="numLikes">loading ...</span> likes
  <button id="likeBtn">like</button>
  <script>
   let numLikes = 0;
   // not shown: like button code
   function handleData(data) {
     numLikes = data.numLikes;
     const n = document.getElementById('numLikes');
     n.textContent = numLikes;
   }
   const dataPromise = fetchJSON("/api/v1/likes/");
   dataPromise.then(handleData);
  </script>
</body></html>
```

1. GET request to REST API
2. When response arrives, calls `handleData(data)`
3. `handleData()` updates `numLikes` and updates the DOM

Simplification

- `fetchJSON` is a simplified version of the Fetch API
  - Calls built-in fetch()
  - Calls built-in json()

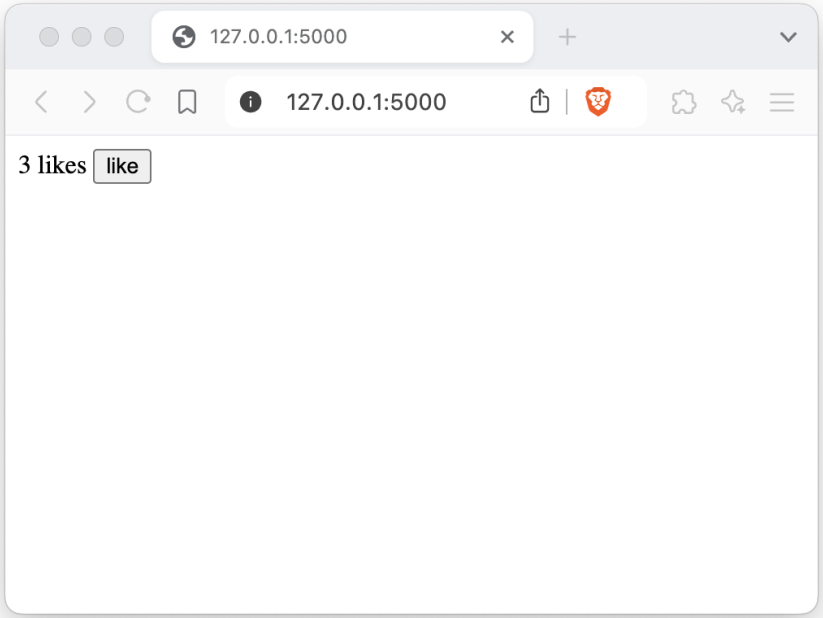
```
function fetchJSON(url) {
  return fetch(url)
    .then(response => {
      return response.json();
   });
}
```

Client perspective and server perspective

- Server sees two requests
  - Initial page load (index.html)
  - REST API request

```
GET / HTTP/1.1 200
GET /api/v1/likes/ HTTP/1.1 200
```

- Client has corrected initial num likes



Fetch timing diagram

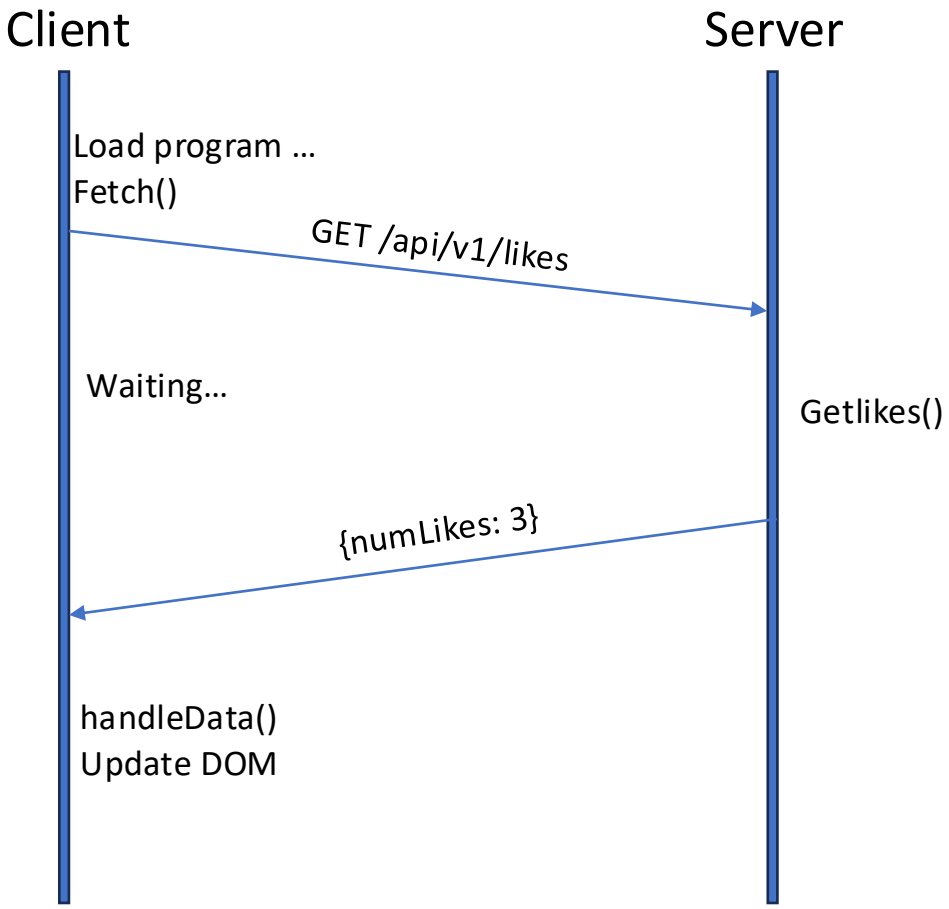
```
let numLikes = 0;
function handleData(data) {
  numLikes = data.numLikes;
  const n = document.getElementById('numLikes');
  n.textContent = numLikes;
}
const dataPromise = fetchJSON("/api/v1/likes/");
dataPromise.then(handleData);
```

```
LIKES = 3
@app.route('/api/v1/likes/')
def get_likes():
    """Return num likes."""
    context = {"numLikes": LIKES}
    return flask.jsonify(context), 200
```

Fetch timing diagram

```
let numLikes = 0;
function handleData(data) {
  numLikes = data.numLikes;
  const n = document.getElementById('numLikes');
  n.textContent = numLikes;
}
const dataPromise = fetchJSON("/api/v1/likes/");
dataPromise.then(handleData);
```

```
LIKES = 3
@app.route('/api/v1/likes/')
def get_likes():
    """Return num likes."""
    context = {"numLikes": LIKES}
    return flask.jsonify(context), 200
```



What does `fetch` return?

```
<html><body>
  <span id="numLikes">loading ...</span> likes
  <button id="likeBtn">like</button>
  <script>
   let numLikes = 0;
   // not shown: like button code
   function handleData(data) {
     numLikes = data.numLikes;
     const n = document.getElementById('numLikes');
     n.textContent = numLikes;
   }

   const dataPromise = fetchJSON("/api/v1/likes/");
   dataPromise.then(handleData);
  </script>
</body></html>
```

- The fetch API returns a *Promise*
- Why not return the data?
  - Network requests are slow
  - Everything else would have to wait

Agenda

- Review
- Client/server synchronization (read)
- Fetch API
- **Promises**
- Anonymous functions
- Client/server synchronization (write)

Promises

- A *Promise* represents a value that will be available in the future
- `fetchJSON()` sends a request
- `fetchJSON()` returns a Promise

```
const dataPromise = fetchJSON("/api/v1/likes/");
```

Promise handler (callback)

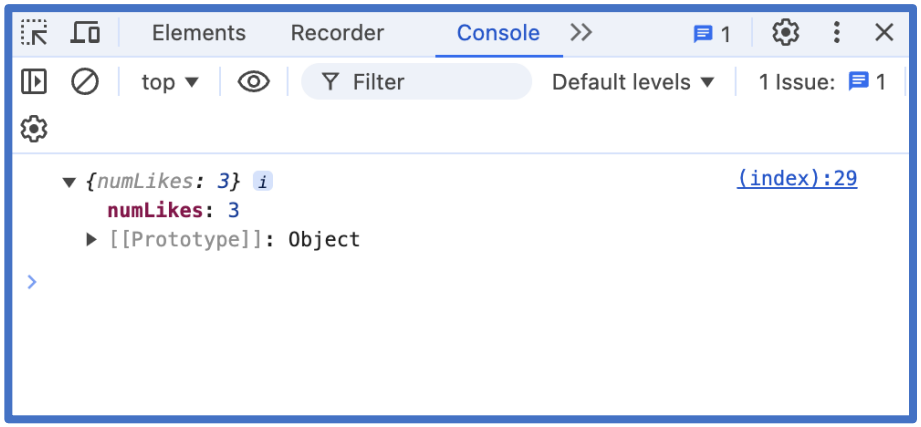
- Promise *handler* runs when Promise is completed
  - AKA callback
- **Handler is a function** provided by `.then()`

```
function handleData(data) {
  numLikes = data.numLikes;
  const n = document.getElementById('numLikes');
  n.textContent = numLikes;
}

const dataPromise = fetchJSON("/api/v1/likes/");
dataPromise.then(handleData)
```

Promise handler input

- Promise handler **input** is the Promise's **value**

```
function handleData(data) {
  numLikes = data.numLikes;
  const n = document.getElementById('numLikes');
  n.textContent = numLikes;
}

const dataPromise = fetchJSON("/api/v1/likes/");
dataPromise.then(handleData)
```



Refactor question

- If we refactor, will the output change?

```
function handleData(data) {
  numLikes = data.numLikes;
  const n = document.getElementById('numLikes');
  n.textContent = numLikes;
}

const dataPromise = fetchJSON("/api/v1/likes/");
dataPromise.then(handleData)
```

```
function handleData(data) {
  numLikes = data.numLikes;
  const n = document.getElementById('numLikes');
  n.textContent = numLikes;
}

fetchJSON("/api/v1/likes/").then(handleData);
```

Refactor solution

- If we refactor, will the output change?
- No. The handler runs when the response arrives.

```
function handleData(data) {
  numLikes = data.numLikes;
  const n = document.getElementById('numLikes');
  n.textContent = numLikes;
}

const dataPromise = fetchJSON("/api/v1/likes/");
dataPromise.then(handleData)
```

```
function handleData(data) {
  numLikes = data.numLikes;
  const n = document.getElementById('numLikes');
  n.textContent = numLikes;
}

fetchJSON("/api/v1/likes/").then(handleData);
```

Agenda

- Review
- Client/server synchronization (read)
- Fetch API
- Promises
- **Anonymous functions**
- Client/server synchronization (write)

Anonymous function motivation

- This callback function is use only once

```
function handleData(data) {
  numLikes = data.numLikes;
  const n = document.getElementById('numLikes');
  n.textContent = numLikes;
}
fetchJSON("/api/v1/likes/").then(handleData);
```

Anonymous functions

- Refactor to use *anonymous functions,* or functions without a name
- AKA lambda function, AKA function literal
- AKA JavaScript **arrow function**

```
function handleData(data) {
  numLikes = data.numLikes;
  const n = document.getElementById('numLikes');
  n.textContent = numLikes;
}
fetchJSON("/api/v1/likes/").then(handleData);
```

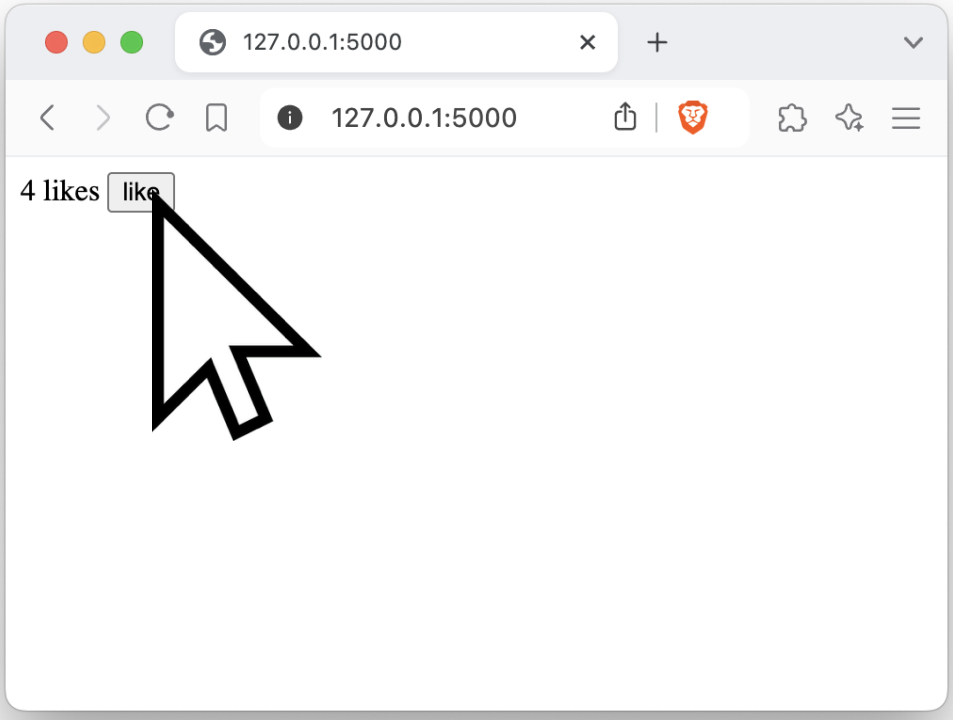
```
fetchJSON("/api/v1/likes/").then((data) => {
  numLikes = data.numLikes;
  const n = document.getElementById('numLikes');
  n.textContent = numLikes;
});
```

Agenda

- Review
- Client/server synchronization (read)
- Fetch API
- Promises
- Anonymous functions
- **Client/server synchronization (write)**

Another synchronization problem

- What happens is the user clicks the "like" button?
- **Problem**: Client and server are out of sync again
- **Solution**: Write to the REST API



REST API Write

- Support PUT request in the REST API
  - Overwrite entire value
  - Return copy of value

```
LIKES = 3  # HACK "Database"

@app.route('/api/v1/likes/', methods=["PUT"])
def put_likes():
    """Set num likes and return num likes."""
    global LIKES
    LIKES = flask.request.json["numLikes"]
    context = {"numLikes": LIKES}
    return flask.jsonify(context), 200
```

```
$ PUT localhost:8000/api/v1/likes/ numLikes=4

{
    "numLikes": "4"
}
```

Exercise

- Where should we change our code to write to the REST API?

```
<html><body>
  <span id="numLikes">loading ...</span> likes
  <button id="likeBtn">like</button>

  <script>
   let numLikes = 0;
   function addLike() {
     numLikes += 1;
     const n = document.getElementById('numLikes');
     n.textContent = numLikes;
   }
   const btn = document.getElementById("likeBtn");
   btn.addEventListener("click", addLike);

   fetchJSON("/api/v1/likes/").then((data) => {
     numLikes = data.numLikes;
     const n = document.getElementById('numLikes');
     n.textContent = numLikes;
   });
  </script>
</body></html>
```

Solution

- **`addLike()`** calls **`fetch`** with new API route

```
function addLike() {
    fetch("/api/v1/likes/", {
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ "numLikes": numLikes }),
    }).then((data) => {
      numLikes += 1;
      const n = document.getElementById('numLikes');
      n.textContent = numLikes;
    });
  }
```

Summary

- *Client and server synchronization* is needed when they share data
  - E.g., the number of likes
- The *Fetch API* is a JavaScript function for making HTTP requests
- A *Promise* represents a value that will be available in the future
- A *Promise handler* runs when Promise is fulfilled
- *Anonymous functions* are functions without a name